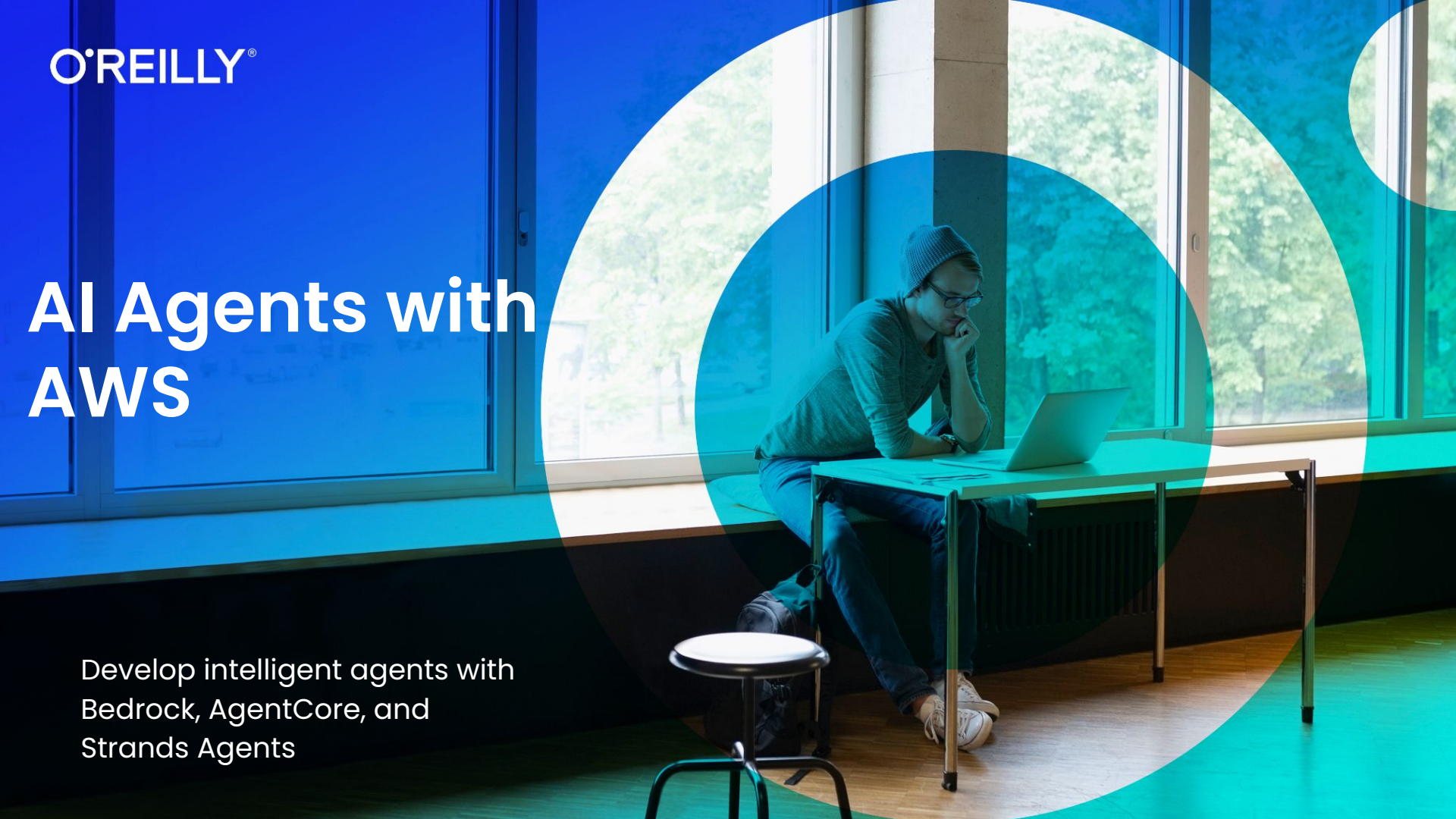
O'REILLY®

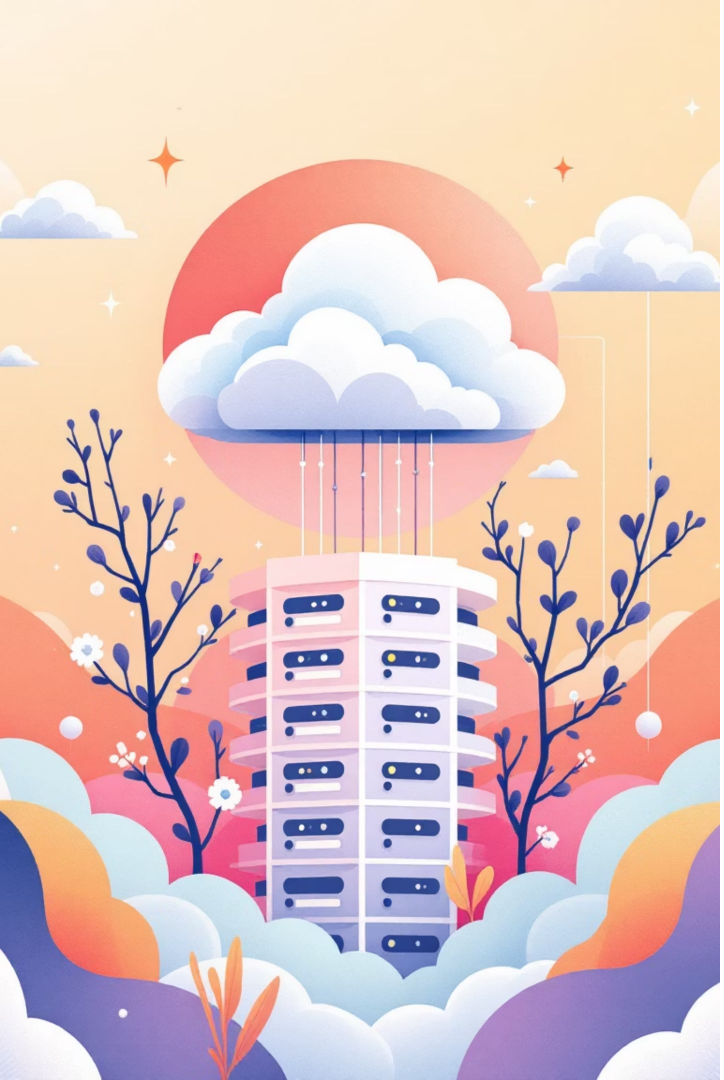
AI Agents with AWS

Develop intelligent agents with Bedrock, AgentCore, and Strands Agents



About Me

- Senior Cloud AI Architect at DoiT.
- 10+ years experience in AWS, cloud architecture, and machine learning.
- Expert in RAG system design, optimization, and responsible AI practices.
- Developed 20+ production ready solution
- Goal: Helping enterprises deploy production-ready, grounded GenAI applications.



Building Production-Ready AI Agents on AWS

Welcome to this hands-on workshop where you'll learn to design, build, and deploy intelligent AI agents using AWS's cutting-edge infrastructure. Over the next 4.5 hours, you'll work through 5 comprehensive sections that take you from foundational concepts to production deployment.

1

Workshop Duration

4 hours of intensive hands-on learning

2

Hands-On Sections

5 practical exercises with real-world applications

3

Live Deployment

Deploy functioning agents directly to AWS infrastructure

What you'll learn

- Build and deploy agents in AWS using AgentCore and Strands Agents
- Integrate LLMs (AWS Bedrock, Anthropic, or Amazon Nova) into agent workflows
- Implement strand-based orchestration, agent memory, toolchains, and context
- Monitor, secure, and scale AgentCore/Strands Agents deployments in production

What Are AI Agents?

AI agents represent a fundamental shift from traditional application architecture. Instead of following predetermined logic paths, they reason dynamically through problems, making intelligent decisions based on context and available tools.

1

Traditional Application

User Input → Fixed Logic → Output

Predetermined workflows with no flexibility

2

AI Agent

User Input → LLM Reasoning → Tool Selection → Action → Output

Dynamic decision-making with adaptive responses

> **Key Difference:** Agents make dynamic decisions based on context, reasoning through multi-step problems without predetermined workflows. They adapt in real-time to solve complex challenges that would require extensive coding in traditional systems.

Problems Agents Solve

AI agents excel at handling sophisticated scenarios that challenge traditional automation approaches. Here's where they provide the most value:

Complex Multi-Step Tasks

"Book a flight, hotel, and rental car for my trip"

Agents coordinate multiple services and handle dependencies automatically

Dynamic Environments

Adapt to changing conditions without reprogramming

Real-time responses to unexpected situations or data changes

Personalization at Scale

Understand context and user preferences

Tailor experiences to individual needs across thousands of users

Automation with Judgment

Handle edge cases that rules can't cover

Apply reasoning to ambiguous scenarios requiring nuanced decisions

Agent Architecture

Understanding the three core components that power every AI agent is essential for building production-ready systems. This architecture follows the ReAct (Reasoning + Acting) pattern, the industry standard for agentic AI systems.

Model (Reasoning Engine)

The LLM (Claude/Nova) that processes requests, reasons through problems, and determines the best course of action based on available context and tools.

Tools (Actions/Capabilities)

Functions the agent can invoke—APIs, databases, calculators, web searches, or custom integrations that extend the agent's capabilities beyond text generation.

Prompt (Instructions/Context)

System instructions that define the agent's role, behavior rules, constraints, and contextual knowledge. This guides how the agent applies its reasoning.

Agent = Model + Tools + Prompt

This simple formula represents the complete architecture for building intelligent, autonomous systems that can reason and act.

When to Use Agents

Making the right architectural choice is critical. Agents aren't always the answer—here's how to decide when they're the right tool for the job.

Use Agents When:

Task requires reasoning

Problems need interpretation and judgment

Multiple steps needed

Workflows involve sequential or branching operations

Dynamic decision-making

Actions depend on runtime context and conditions

Tool selection varies by context

Different situations require different tools

Don't Use Agents When:

Simple CRUD operations

Basic database interactions don't need reasoning

Fixed workflow sufficient

Predetermined logic paths handle all cases

No external actions needed

Pure text generation without tool invocation

Deterministic logic required

Exact, predictable outputs are mandatory

Framework Comparison

AWS offers multiple approaches for building AI agents. Understanding the tradeoffs helps you choose the right tool for your production needs.

Feature	Strands	AgentCore	Bedrock Agents (Legacy)
Approach	Model-driven SDK	Production Platform	Managed service
Control	High	High	Medium
Deployment	Flexible	AWS-native	Full abstraction
Session Mgmt	Manual	Built-in	Built-in
Memory	External	Short + Long-term	Short-term
Best For	Custom workflows	Enterprise agents	Quick prototyping
Status	Active (2025+)	GA (Oct 2025)	Legacy

☆ WORKSHOP FOCUS

Strands SDK for development + **AgentCore Platform** for production deployment

AgentCore launched in general availability October 2025 and represents the evolved, production-ready platform for agentic AI on AWS. Bedrock Agents remains available but is the legacy approach.

AgentCore Platform

AgentCore is AWS's purpose-built infrastructure for production AI agents. Unlike retrofitted solutions, it's designed from the ground up for agentic workloads with enterprise-grade security and cost efficiency.

Runtime
Serverless execution with 8-hour maximum session duration and configurable idle timeout. No infrastructure management required.

Gateway
Secure tool access via Model Context Protocol (MCP). Converts APIs, Lambda, and AWS services into MCP-compatible tools.

Memory
Dual-layer system: short-term context for current sessions plus long-term insights via vector embeddings across sessions.

Identity
OAuth and IAM authentication with fine-grained access control for enterprise security compliance.

Observability
OpenTelemetry tracing with native CloudWatch integration for real-time monitoring and debugging.

Code Interpreter
Isolated Python environment for agent-generated code execution with no sandbox escape risk.

Browser
Managed web automation instances for agents requiring browser-based interactions and scraping.

Strands Agents SDK

Strands takes a model-driven approach that minimizes boilerplate and maximizes flexibility. The LLM handles orchestration while you define tools and behavior through simple, type-safe interfaces.

Model-Driven Approach

LLM (Claude, Nova) handles orchestration logic automatically

Minimal Boilerplate

Write less code, focus on business logic and tool definitions

Flexible Integration

Type-safe tool definitions with schema validation

Example Implementation (Python):

```
from strands import Agent
from strands.tools import tool

@tool
def calculator(expr: str) -> str:
    """Evaluate a mathematical expression."""
    return str(eval(expr))

agent = Agent(
    system_prompt="You are a helpful math assistant. Solve problems step-by-step.",
    tools=[calculator]
)

response = agent("What is 15 * 23?")
print(response)
```

✨ NEW

TypeScript Available: Strands SDK now supports TypeScript (public preview, Dec 2025) with zod schema validation for type-safe tool definitions.

Poll Time! 🗳️

What use cases brought you here?

Understanding your goals helps us tailor the workshop to your needs. Share your primary use case in the chat!

A. Customer Support Automation

Intelligent chatbots, ticket routing, and automated responses

B. Data Analysis and Reporting

Automated insights, report generation, and data visualization

C. Content Generation

Marketing copy, documentation, and creative content at scale

D. DevOps Automation

Infrastructure management, deployment pipelines, and monitoring

E. Research and Information Gathering

Web scraping, document analysis, and knowledge synthesis

F. Other

Tell us about your unique use case in the chat!

Share in Chat

Exercise Overview

Time to put theory into practice! You'll complete two short exercises that demonstrate the complete agent development and deployment lifecycle.

1

Exercise 1A: Run Base Agent

Duration: 5 minutes

- Create a simple calculator agent using Strands SDK
- Run locally to observe tool invocation in real-time
- Understand the ReAct loop (Reasoning + Acting) in action

2

Exercise 1B: AgentCore Overview

Duration: 5 minutes

- Deploy your agent to AgentCore Runtime infrastructure
- Compare Lambda execution vs. AgentCore Runtime performance
- Understand production deployment path and session management

Let's get started!

Break Time

5 Minute Break ☕

Stretch, grab a coffee, and recharge. When we return, we'll dive into advanced topics that make multi-agent workflows production-ready.

WORKSHOP SECTION

Building Agents with Bedrock and Strands

Master the fundamentals of connecting AWS Bedrock models to the Strands framework and building production-ready agentic tools.

1

Connect Bedrock Models

Configure and integrate Claude and Nova models with Strands

2

Build Custom Tools

Create agent tools that follow best practices and patterns

3

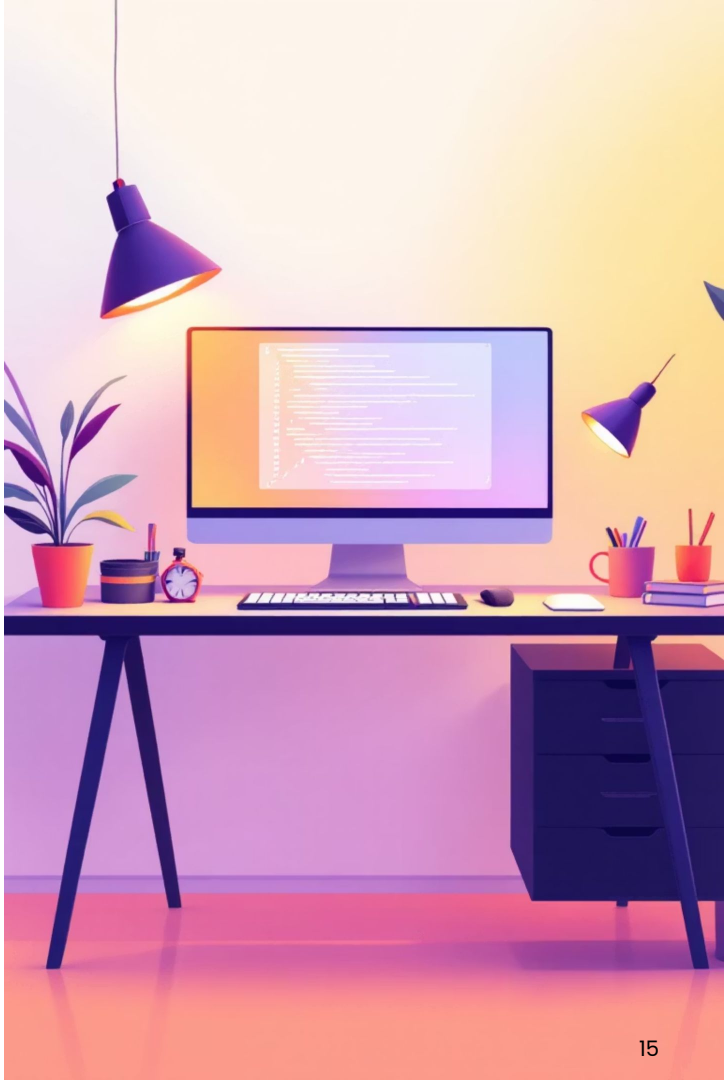
Handle Edge Cases

Implement robust error handling and graceful degradation

4

Test & Deploy

Validate tool invocation and optimize performance



Choosing the Right Bedrock Model

AWS Bedrock offers multiple models optimized for different agentic workloads. Understanding the cost-performance tradeoffs is critical for production deployments.

Model	Input Cost	Output Cost	Speed	Best For
Nova Lite	$0.06/1M	$0.24/1M	⚡⚡⚡	Cost-optimized demos and general tasks
Nova Pro	$0.80/1M	$3.20/1M	⚡⚡	General purpose with complex reasoning
Claude 3.5 Haiku	$0.80/1M	$4.00/1M	⚡⚡	Complex multi-step tasks, tool orchestration
Claude 3.5 Sonnet v2	$6.00/1M	$30.00/1M	⚡	Advanced reasoning, agentic workflows

Workshop Recommendation: Claude 3.5 Haiku

The optimal balance of capability and cost for production agents. Proven in enterprise deployments including Amazon Q and AWS Glue, with excellent tool use and multi-step reasoning capabilities.

Alternative: Nova Lite delivers 85% cost reduction if raw capability is less critical than budget optimization.

Pricing Update (January 2026): Claude 3.5 Haiku represents a 3.2x improvement in value compared to legacy Claude 3 Haiku (2024). All pricing reflects current AWS Bedrock rates.

Connecting Your Agent to Bedrock

Setting up the Bedrock model integration requires just a few lines of code, but configuration choices impact latency, cost, and reproducibility.

```
from strands import Agent
from strands.models import BedrockModel

# Initialize Bedrock model
model = BedrockModel(
    model_id="anthropic.claude-3-5-haiku-20241022-v1:0",
    region="us-east-1"
)

# Create agent with model
agent = Agent(
    model=model,
    system_prompt="You are a helpful assistant",
```

Model Reuse

Initialize once and reuse across multiple agent invocations for efficiency

Region Selection

us-east-1 typically recommended for Bedrock to minimize latency

Version Pinning

Model ID specifies exact version (critical for reproducibility)

Advanced Tool Use (November 2025): Claude now supports orchestrating 20+ tool calls in a single agentic loop—enabled automatically with no code changes. This improves accuracy on complex multi-tool tasks by 5-10%.

Anatomy of a Well-Designed Tool

The quality of your tool implementation directly determines how accurately the LLM will invoke it. Each element serves a specific purpose in guiding the model's decision-making process.

```
@tool
def get_weather(city: str) -> str:
    """Get current weather for a city.

    Args:
        city: City name (e.g., "Seattle", "Toronto")
    Returns:
        JSON string with temperature, conditions, and forecast
    Example:
        >>> get_weather("Seattle")
        '{"temp": 45F, "conditions": "rainy", "forecast": "rainy"}'
    """
    # Implementation
    return weather_data
```

1

Detailed Docstring

The LLM reads this to decide when and how to use the tool

2

Type Hints

Specify parameter types (str, int, List[str], etc.) for validation

3

Descriptive Name

Function name clearly indicates purpose and action

4

Return Annotation

Helps LLM understand and parse output format correctly

5

Concrete Examples

Shows LLM specific usage patterns with real inputs/outputs

6

Error Handling

Graceful degradation prevents agent crashes and maintains flow

> **LLM Perspective:** Tool-selection accuracy is directly proportional to docstring clarity. A vague docstring like "do something" results in misuse, while a precise one like "fetch weather data for a US city and return JSON with temp/conditions" achieves 95%+ correct invocation rates.

Tool Design Principles

DO ✅

- Write detailed docstrings with concrete examples
- Use descriptive parameter names (not x, data, val)
- Return structured data (JSON/dict, not raw strings)
- Handle errors gracefully with informative messages
- Keep tools focused (single responsibility principle)
- Use type hints on all parameters and returns
- Document expected parameter formats and values

DON'T ❌

- Create vague descriptions ("process input", "do work")
- Build multi-purpose tools handling 5+ unrelated tasks
- Let exceptions crash the agent (wrap in try/except)
- Return raw error stack traces to the model
- Require complex setup or hidden dependencies
- Use cryptic names (avoid proc_data, util_func)
- Leave side effects undocumented

Advanced: Bedrock Guardrails Integration

For production agents, consider wrapping tool outputs with Amazon Bedrock Guardrails to filter hallucinations and validate responses before returning to the agent. This adds a critical security layer against prompt injection via tool results.

Tool Implementation Examples

These three patterns cover the most common tool types you'll build: simple computation, external API integration, and database access.

Simple Tool (Single Responsibility)

```
@tool
def calculate(expression: str) -> str:
    """Evaluate a mathematical expression safely.

    Args:
        expression: Math expression (e.g., "15 * 23 + 100")
    Returns:
        Result as string
    Example:
        >>> calculate("15 * 23")
        '345'
    """
    try:
        result = eval(expression)
        return str(result)
    except Exception as e:
        return f"Error: Invalid expression '{expression}'"
```

Tool Implementation Examples

These three patterns cover the most common tool types you'll build: simple computation, external API integration, and database access.

API Tool (External Data)

```
@tool
def get_weather(city: str) -> str:
    """Fetch current weather from external API.

    Args:
        city: US city name (e.g., "Seattle", "New York")
    Returns:
        JSON string with temperature, conditions, humidity
    """
    try:
        response = requests.get(
            f"api.weather.com/current",
            params={"city": city}
        )
        return response.json()
    except requests.RequestException:
        return f"Error: Could not fetch weather for {city}"
```

Tool Implementation Examples

These three patterns cover the most common tool types you'll build: simple computation, external API integration, and database access.

Data Tool (Database Access)

```
@tool
def lookup_user(user_id: str) -> str:
 """Look up user details from database.

 Args:
 user_id: Unique user identifier (e.g.,
"usr_12345")
 Returns:
 JSON string with user name, email, status
 """
 try:
 user = db.query(user_id)
 return json.dumps(user) if user else "User not
found"
 except Exception as e:
 return f"Error: Database lookup failed"
```

Error Handling in Tools

Robust error handling is non-negotiable. Unhandled exceptions crash the agent's reasoning loop and break execution entirely.

❌ Bad Pattern (Will Crash Agent)

```
@tool
def risky_tool(param: str) -> str:
    result = api.call(param)  # Might fail, no
protection
    return result
```

✅ Good Pattern (Graceful Degradation)

```
@tool
def safe_tool(param: str) -> str:
    try:
        result = api.call(param)
        return result
    except ValueError as e:
        return f"Invalid input: {param}. Expected format:
email@domain.com"
    except requests.Timeout:
        return "Error: API timeout. Please try again."
    except Exception as e:
        return "Error: Tool failed unexpectedly. Please
retry."
```

Error Handling in Tools

Best Practice: Specific Error Messages

Generic: "Error occurred"

Specific: "Error: Database connection refused (timeout after 5s). This API requires credentials."

Specific error messages help the LLM decide whether to retry, use an alternative tool, or explain the limitation to the user. Always handle errors inside tools—never let exceptions bubble up to the agent runtime.

Tool Selection Patterns

Understanding when to create a tool versus letting the LLM handle a task is crucial for building efficient agents.

When to Create a Tool ✅

- Access external data (APIs, databases, file systems)
- Perform actions with side effects (send email, create file, update database)
- Execute calculations requiring external services
- Enforce business logic or access control

When NOT to Create a Tool ❌

- Simple text formatting (LLM can do this perfectly)
- Basic reasoning or logic (LLM's core strength)
- Tasks requiring human judgment (ask user instead)
- Information retrieval from system prompt context

Rule of Thumb
If it needs external data or has side effects, make it a tool. If it's pure computation or reasoning, let the LLM handle it.

Tool Selection Patterns

Tool Granularity Guidance

Too Coarse
One "do_everything" tool ← Don't do this

Sweet Spot
Single-purpose tools (5-10 total)

Too Fine
50 micro-tools ← Confuses model, slow selection

Example Decision

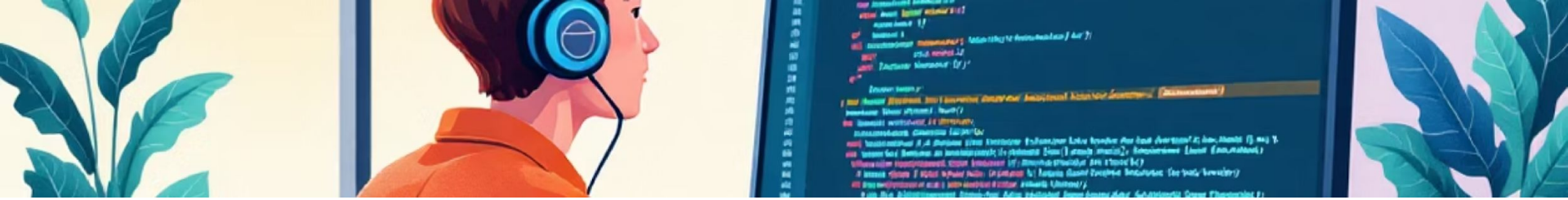
> **Agent asks:** "Book me a flight from Seattle to Toronto"

Tools Needed ✅

- search_flights(from, to, date) ← External data
- check_price_limits(price) ← Business logic
- book_flight(flight_id) ← Action with side effect

Not Needed ❌

- compare_prices() ← LLM can reason
- explain_flight_options() ← LLM can explain
- format_flight_details() ← LLM can format



LIVE DEMO

Building Your First Tools

Watch as we create three production-ready tools from scratch, demonstrating the patterns and principles covered in this section.

Weather API Tool

External data retrieval demonstrating error handling and API integration patterns

Calculator Tool

Safe computation showcasing input validation and try/except patterns

Text Analyzer Tool

Data transformation with structured JSON output and single responsibility design

Watch For

- @tool decorator usage and docstring placement
- Type hints on parameters and return values
- Try/except blocks wrapping external calls
- Informative error messages for debugging

Code Repository

All demo code will be available at:

section2/demo_tools.py

Live Demo: Recap & Focus Points

Before we dive into the code, let's quickly recap what we'll be building and the key concepts to observe.

Weather API Tool

External data retrieval, error handling, and API integration.

Calculator Tool

Safe computation with input validation and robust error patterns.

Text Analyzer Tool

Data transformation, structured output (JSON), and single responsibility.

Decorator Usage

Proper placement of `@tool` and docstrings.

Type Hints

Clarity for parameters and return values.

Error Handling

`try/except` blocks for external calls.

Informative Errors

Messages for easier debugging.

Access all code examples for this demo at: `section2/demo_tools.py`

Discussion

1

How do you decide what should be a tool vs. what the LLM handles?

Hint: External data or side effects = tool

Discussion

2

What's the right number of tools for an agent?

Hint: 5-10 is sweet spot; more creates selection overhead

Discussion

3

How do you handle tools that might fail?

Hint: Try/except, informative error messages

Discussion

4

When should you split a tool into multiple tools?

Hint: When it violates the single responsibility principle

Best Practices for Robust Tooling

To ensure your agent tools are effective, reliable, and production-ready, adhere to these essential guidelines.

Tool Design Essentials

- Crystal-clear docstrings (LLM uses these for decisions)
- Type hints on every parameter and return value
- Single responsibility per tool (do one thing well)
- Graceful error handling with specific messages
- Examples in docstrings showing expected usage

System Prompt Integration

- Describe when and why to use each tool
- Provide context about tool limitations
- Set expectations for tool behavior
- Example: "Use search_orders only for customer queries, not inventory"

Testing Checklist

- Test each tool independently (unit test style)
- Test with various query types and edge cases
- Test error scenarios (invalid input, API timeout)
- Measure tool response latency (should be <2s typically)
- Verify structured output format (JSON consistency)

Production Deployment Readiness

- Add logging to tools for observability
- Consider Bedrock Guardrails for output validation
- Implement tool result caching for expensive operations
- Monitor tool success/failure rates in CloudWatch
- Document tool rate limits and costs

Key Takeaways: Robust Tooling

Summarizing the essential best practices for building effective and reliable agent tools.

Model Selection

Align models (e.g., Claude 3.5 Haiku) with task complexity and cost for optimal performance.

Clear Docstrings

Ensure precise and comprehensive docstrings for accurate LLM tool selection and usage.

Error Handling

Implement robust error handling within tools to prevent agent crashes and provide clear feedback.

Single Responsibility

Design tools with a single purpose; aim for 5-10 focused tools per agent.

Thorough Testing

Test tools independently and with diverse agent queries, including edge cases.

Advanced Tool Use

Leverage the advanced capabilities for agents making 20+ tool calls per loop.

Break Time

5 Minute Break ☕

Stretch, grab a coffee, and recharge. When we return, we'll dive into advanced topics that make multi-agent workflows production-ready.

INTRODUCTION

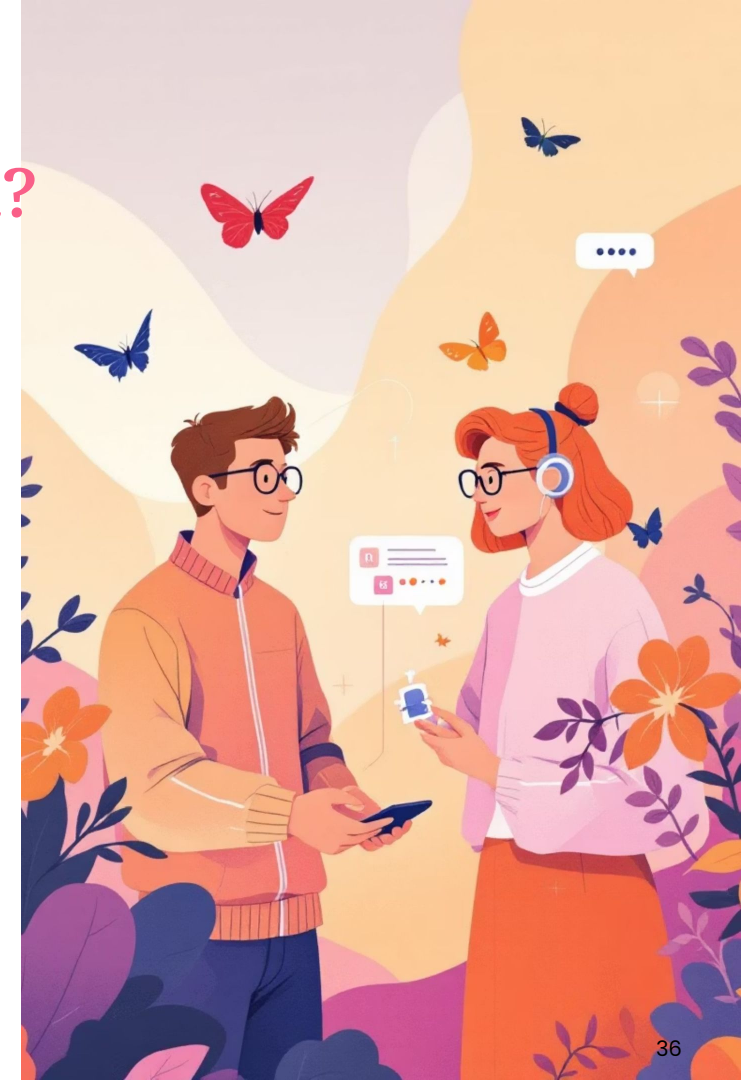
Why Multi-Agent Orchestration?

Single Agent Limitations

- Attempts to handle every task with one monolithic prompt
- Complex instructions become unmaintainable over time
- Debugging is challenging due to opaque reasoning
- Scalability bottlenecks emerge as workload increases

Multi-Agent Benefits

- Specialization enables domain-specific expertise per agent
- Modularity simplifies testing, maintenance, and versioning
- Scalability through parallel execution and independent scaling
- Transparency via visible decision flows and handoffs
- Resilience with failure isolation—one agent failure doesn't cascade



STRANDS SDK

Three Orchestration Patterns in Strands

Strands SDK provides three distinct orchestration patterns, each optimized for different workflow requirements and control models.

Graph Pattern

Control: Developer-defined DAG with LLM routing

Flexibility: High—conditional branching and decision trees

Best For: Dynamic workflows with conditional logic

Example: If ticket is urgent → escalate; else → queue for standard processing

Swarm Pattern

Control: Agents hand off autonomously to each other

Flexibility: Very High—emergent collaboration

Best For: Complex problem-solving requiring agent autonomy

Example: Research team self-organizes to tackle multi-faceted problems

Workflow Pattern

Control: Pre-defined deterministic DAG

Flexibility: Low—fixed execution path

Best For: Repeatable processes with parallel tasks

Example: ETL pipeline: Extract data → Transform → Load into database

Workshop Focus: We'll concentrate on the Graph pattern—the most commonly used and most flexible for learning multi-agent orchestration concepts.

FLOW TOPOLOGIES

Sequential, Parallel, and Conditional Flows

Within the Graph pattern, you can design different execution topologies to match your workflow needs.

Sequential Flow

Linear execution: A → B → C → Result

Use Case: When each step depends on the previous output

Example: Research phase → Analysis phase → Format final report

FLOW TOPOLOGIES

Sequential, Parallel, and Conditional Flows

Within the Graph pattern, you can design different execution topologies to match your workflow needs.

Parallel Flow

Concurrent execution: A fans out to B, C, D, then aggregates to E

Use Case: When tasks are independent and can run simultaneously

Example: Fetch data from three APIs in parallel, then aggregate results

FLOW TOPOLOGIES

Sequential, Parallel, and Conditional Flows

Within the Graph pattern, you can design different execution topologies to match your workflow needs.

Conditional Flow

Branching logic: A routes to either B_1 or B_2 based on conditions

Use Case: When workflow path depends on runtime decisions

Example: Triage urgent support tickets to priority queue vs. standard processing

FLOW TOPOLOGIES

Sequential, Parallel, and Conditional Flows

Within the Graph pattern, you can design different execution topologies to match your workflow needs.

Cyclic with Loop Control

Iterative execution: A → B → C, with feedback loop back to B if needed

Use Case: When tasks require refinement or iteration

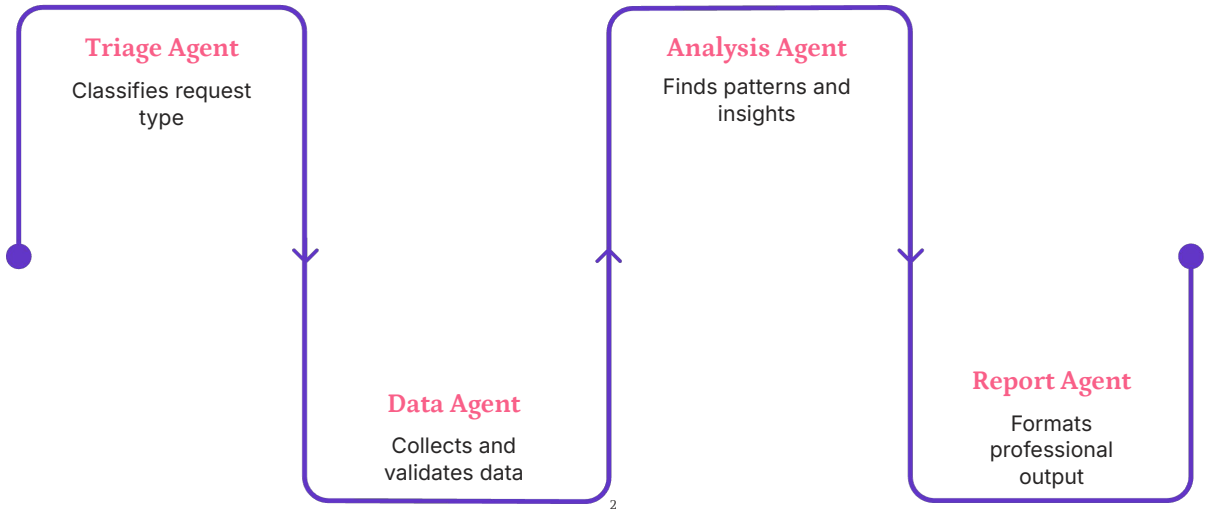
GraphBuilder supports cyclic graphs with max iteration limits to prevent infinite loops

Visualizing Multi-Agent Flows

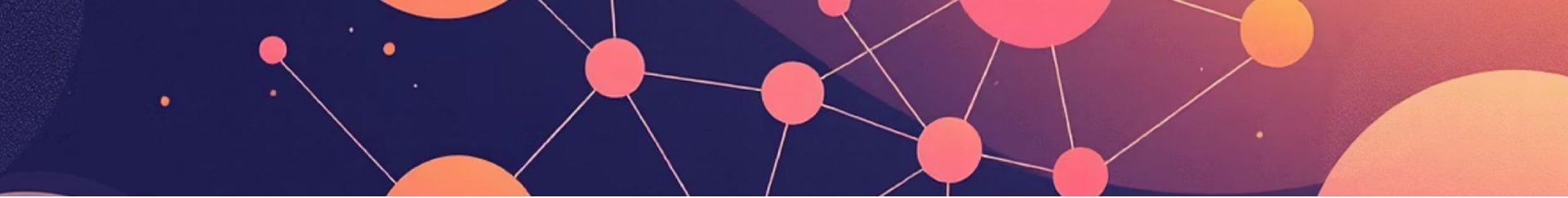
Let's trace a complete workflow from user query to final output, showing how agents collaborate and memory persists across handoffs.

Complete Workflow Example

User Query: "Analyze this sales data and create a professional report"



Each agent maintains a single, well-defined responsibility. Memory persists across all handoffs, and conditional routing enables intelligent workflow decisions.



CORE CONCEPTS

GraphBuilder Core Concepts

Building multi-agent workflows in Strands requires understanding three fundamental components that work together to orchestrate agent collaboration.

1

Nodes: Individual Agents

Each node represents a specialized agent with a distinct role in your workflow.

```
builder.add_node(research_agent, "researcher")
builder.add_node(analysis_agent, "analyst")
builder.add_node(report_agent, "writer")
```

2

Edges: Data Flow Between Agents

Edges define how data and control flow from one agent to the next.

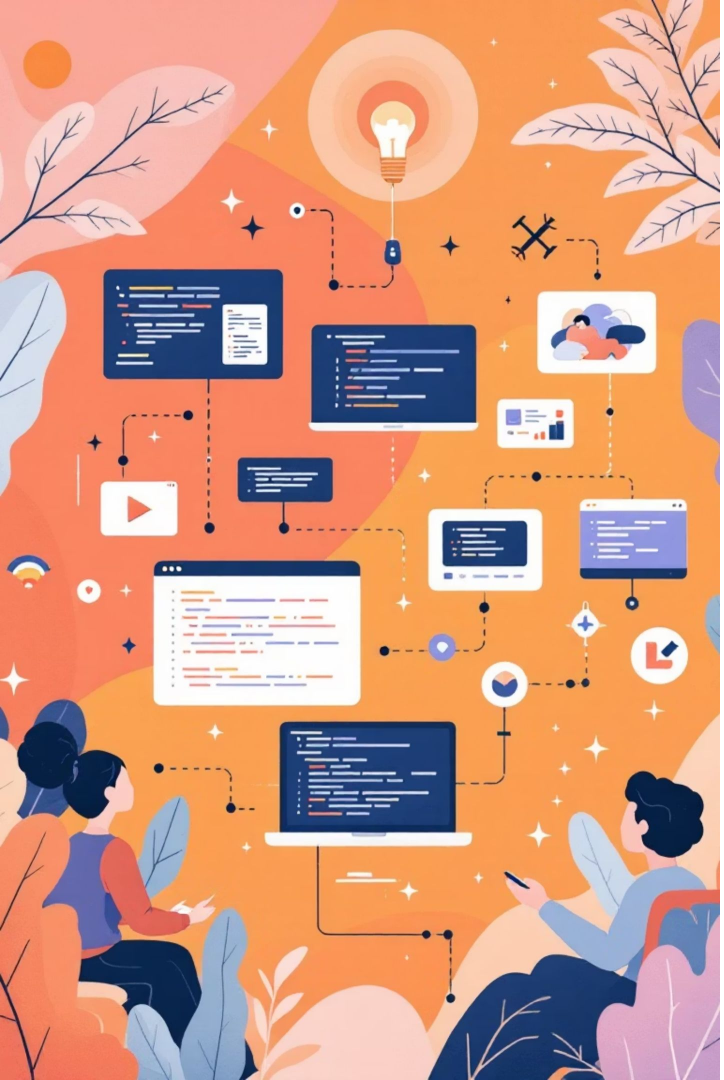
```
# Simple edge
builder.add_edge("researcher", "analyst")

# Conditional edge with routing logic
def should_escalate(result):
    return "urgent" in result.text.lower()

builder.add_conditional_edge(
    "analyst", should_escalate,
    {"true": "escalation", "false": "standard"}
)
```

3

Entry Point: Workflow Start

The entry point designates which agent receives the initial user input.

```
builder.set_entry_point("researcher")
```



IMPLEMENTATION

Demo

RESULTS API

AgentResult vs GraphResult

Understanding the difference between individual agent outputs and complete workflow results is crucial for debugging and accessing intermediate steps.

AgentResult

Scope: Output from a single agent execution

Contains:

- Text response generated by the agent
- Tool calls made during execution
- Agent-specific metadata

Access Pattern:

```
agent_result.text
agent_result.tool_calls
agent_result.metadata
```

Passed as input to the next agent in the workflow chain.

GraphResult

Scope: Output from entire multi-agent workflow

Contains:

- All AgentResults from executed nodes
- Complete execution path taken
- Timing and token usage metrics
- Cost estimation (if enabled)

Access Pattern:

```
result.text  # Final output
result.results["step1"]
result.execution_time
result.token_usage
result.cost
result.execution_path
```

```
# Example: Check workflow execution path
if "escalation_team" in result.execution_path:
 print("Case was escalated")
 reason = result.results['escalation_team'].text
 print(f"Reason: {reason}")
```

HANDS-ON EXERCISE

Exercise Part A: Build Your First Multi-Agent Workflow

Choose one use case and build a 3-agent sequential workflow. Each agent should have a distinct role and build upon the previous agent's output.

Content Pipeline

Flow: Ideas Agent → Draft Agent → Edit Agent

- Ideas agent brainstorms content topics and outlines
- Draft agent writes the first draft based on ideas
- Edit agent polishes, formats, and finalizes content

Data Analysis

Flow: Collect Agent → Analyze Agent → Visualize Agent

- Collect agent gathers data from multiple sources
- Analyze agent identifies patterns and insights
- Visualize agent creates summary with key metrics

Customer Service

Flow: Classify Agent → Process Agent → Respond Agent

- Classify agent categorizes incoming support ticket
- Process agent gathers relevant context and history
- Respond agent drafts personalized customer response

Requirements

- Create three agents, each with distinct system prompts and clear purpose
- Implement sequential flow (Agent A → Agent B → Agent C)
- Ensure each agent has access to previous agent's output
- Test your workflow with a realistic input query

Time Allocation: 20 minutes | **Template:** section3/exercise_part_a.py

Break Time

10 Minute Break ☕

Stretch, grab a coffee, and recharge. When we return, we'll dive into advanced topics that make multi-agent workflows production-ready.

Memory Management

Learn how to maintain context and state across agents and sessions

Error Handling

Implement resilience strategies for robust production workflows

Production Readiness

Best practices for deploying and monitoring multi-agent systems

MEMORY SYSTEMS

Memory in Multi-Agent Workflows

Effective memory management enables agents to maintain context, personalize interactions, and learn from past experiences across workflows and sessions.

Short-Term Memory

Session Context

- Scoped to current conversation only
- Stored in session manager during workflow execution
- Available to all agents within the active workflow
- Automatically cleared when session ends

Examples: User preferences, request classification, partial results, conversation state

Long-Term Memory

AgentCore Memory (Production)

- Persists across sessions and multiple users
- Shared between agents and workflows
- Stored in vector database with semantic search
- Learns patterns from historical interactions

Examples: Customer history, common issue patterns, organizational best practices, learned solutions

Why Memory Matters

Context Awareness

Each agent understands the full conversation context and can make informed decisions based on complete information.

Personalization

Agents adapt their behavior based on user history, preferences, and past interactions for tailored experiences.

Efficiency

Avoid redundant processing by leveraging previously computed results and learned patterns from similar queries.

Continuity

Multi-turn conversations work seamlessly with consistent context maintained across agent handoffs and session boundaries.

Managing Sessions in Multi-Agent Workflows

Effective session management is critical for agents to maintain state, context, and memory across interactions. Strands provides flexible session managers for both local development and robust production environments.

Local Development: `FileSessionManager`

Ideal for rapid iteration and testing, this manager stores session data directly on the local filesystem. It's simple to set up but not suitable for shared or persistent memory needs in production.

```
from strands.session import FileSessionManager

session_manager = FileSessionManager(
    session_dir="./sessions"
)

agent = Agent(
    model=model,
    system_prompt="You remember all conversations",
    tools=[...],
    session_manager=session_manager
)

# Use with session ID
response = agent("My name is Alice", session_id="user_123")
response = agent("What's my name?", session_id="user_123")
# Output: "Your name is Alice"
```

Managing Sessions in Multi-Agent Workflows

Production Deployment:

`AgentCoreMemorySessionManager`

The recommended solution for production, enabling persistent long-term memory across sessions and agents by integrating with AgentCore's vector database. This manager supports rich context, personalization, and retention policies.

The **key distinction** lies in persistence and scalability: `FileSessionManager` is for local, ephemeral storage, while `AgentCoreMemorySessionManager` offers robust, shared, and long-term memory for complex production use cases.

```
from strands.session import AgentCoreMemorySessionManager

# Production: Use AgentCore Memory for shared, persistent memory
session_manager = AgentCoreMemorySessionManager(
    session_id="session_abc789",
    actor_id="customer_xyz123",
    region_name="us-east-1",
    load_long_term_memories=True,
    short_term_retention_days=7,
    long_term_retention_days=365
)

agent = Agent(
    model=model,
    system_prompt="""You are a customer service agent.
    You have access to customer history and context.
    Personalize responses based on their preferences.""",
    tools=[...],
    session_manager=session_manager
)

# Works across multiple interactions and agents
response1 = agent("Book me a flight", session_id="session_abc789")
response2 = agent("Use my preferred airline", session_id="session_abc789")
# Agent remembers: Customer's preferred airline from history
```

Robust Error Handling Strategies

Building resilient multi-agent workflows requires thoughtful error handling. Here are some strategies to ensure your systems remain stable and deliver reliable outcomes, even when unexpected issues arise.

1

Retry with Exponential Backoff + Jitter

When to use: Network timeouts, rate limits, transient API errors.

Why jitter: Prevents "thundering herd" (multiple agents retrying simultaneously).

```
import random
import time

def call_with_retry(api_call, max_attempts=3):
    for attempt in range(max_attempts):
        try:
            return api_call()
        except Exception as e:
            if attempt < max_attempts - 1:
                # Exponential backoff: 1s, 2s, 4s + random jitter
                wait_time = (2 ** attempt) + random.uniform(0, 1)
                time.sleep(wait_time)
            else:
                raise
```

2

Fallback Chain (Multiple Data Sources)

When to use: Service redundancy, multiple data sources, high availability.

Example: Try live API → cached API → local database.

```
def get_user_data(user_id):
    try:
        return primary_api(user_id)
    except:
        try:
            return backup_api(user_id)
        except:
            return cached_data(user_id)
```

Robust Error Handling Strategies

Building resilient multi-agent workflows requires thoughtful error handling. Here are some strategies to ensure your systems remain stable and deliver reliable outcomes, even when unexpected issues arise.

3

Graceful Degradation (Partial Results)

When to use: Non-critical data, user experience more important than completeness.

Example: Show basic profile info even if recommendations fail.

```
def get_user_profile(user_id):
    try:
        full_profile = fetch_full_profile(user_id)
        return full_profile
    except:
        # Return partial profile if full fetch fails
        return fetch_basic_profile(user_id)
```

4

Human Escalation (Complex Cases)

When to use: Edge cases, high-value transactions, ambiguous requests.

Example: E-commerce order >$10k or complex support issue.

```
@tool
def escalate_to_human(reason: str) -> str:
    """Hand off to human agent for complex issue."""
    ticket_id = create_support_ticket(reason)
    notify_human_agent(ticket_id)
    return f"Escalated to human (ticket {ticket_id})"
```

Robust Error Handling Strategies

Building resilient multi-agent workflows requires thoughtful error handling. Here are some strategies to ensure your systems remain stable and deliver reliable outcomes, even when unexpected issues arise.

5

Circuit Breaker (Prevent Cascading Failures)

When to use: External service is down/degraded, prevent cascading failures.

Example: If payment API fails 5x, return cached response instead.

```
class CircuitBreaker:
    def __init__(self, failure_threshold=5):
        self.failure_count = 0
        self.failure_threshold = failure_threshold
        self.is_open = False

    def call(self, func):
        if self.is_open:
            return self.fallback()

        try:
            result = func()
            self.failure_count = 0  # Reset on success
            return result
        except:
            self.failure_count += 1
            if self.failure_count >= self.failure_threshold:
                self.is_open = True
            raise

    def fallback(self):
        return cached_response()
```

Choosing the Right Error Pattern

Selecting the appropriate error handling strategy is crucial for building robust multi-agent systems. This guide helps you map common error types to effective patterns, ensuring your applications remain resilient and provide a consistent user experience.

Error Handling Strategies Overview

Network timeout (transient)	Retry + Backoff	Service will likely recover; retry should succeed after a short delay.
API rate limited	Retry + Backoff with jitter	Allows rate limit to reset and prevents simultaneous retries from overloading the service.
Service down (persistent)	Circuit Breaker	Prevents hammering a failed service, allowing it time to recover, and can return cached responses.
Authentication failure	Escalate or Error	Not a transient issue; requires user action or human intervention to resolve.
Missing optional data	Graceful Degradation	Provides a partial but functional experience; some data is better than no data.
Complex user request	Human Escalation	Beyond automated agent capability; requires human intelligence for nuanced interpretation or action.
Multiple data sources	Fallback Chain	Ensures data availability by attempting primary, secondary, and cached sources in sequence.

Decision Tree for Error Resolution

Use this flow to quickly determine the most effective error handling pattern for your situation:

1

Error Detected

An unexpected issue has occurred within the system.

2

Is the Error Transient?

e.g., network timeout, API rate limit.

If YES: Implement **Retry with Exponential Backoff**.

3

Is it a Persistent Service Outage?

e.g., external service is completely down.

If YES: Engage **Circuit Breaker** and provide **Fallback** options.

4

Is the Affected Data Optional?

e.g., a recommendation engine failure for a user profile.

If YES: Apply **Graceful Degradation**, showing partial results.

5

Does it Require Human Judgment?

e.g., high-value transaction, ambiguous request.

If YES: Initiate **Human Escalation**.

If NO: Return an appropriate error to the user.

Exercise Part B: Memory & Error Handling

It's time to apply what you've learned! Enhance your multi-agent workflow by integrating session management and robust error handling strategies.

Session Management

- Integrate the session manager into your agents.
- Conduct multi-turn conversations to test context retention.
- Verify agents remember relevant information across turns.

Error Handling in Tools

- Identify 2-3 tools prone to failure.
- Implement appropriate error strategies (retry, fallback, graceful degradation).
- Actively test various failure scenarios for each tool.

Testing Resilience

- Simulate an API timeout or service degradation.
- Confirm retry logic and fallback mechanisms activate correctly.
- Verify the agent recovers and proceeds gracefully.

Success Criteria

- Agents successfully utilize the session manager.
- Multi-turn conversations maintain full context.
- Error handling prevents workflow crashes.
- Appropriate strategy used for each failure type.

Time: 20 minutes

Template: `section3/exercise_part_b.py`

Production Error Patterns: Real-World Examples

Effective error handling is critical for maintaining system reliability and a seamless user experience. Here are practical applications of various error patterns in common system architectures.

E-Commerce Platform

✅ Retry payment processing (3 attempts with backoff)
✅ Fallback to backup payment processor if primary fails
✅ Circuit breaker on inventory service (if down, show cached inventory)

✅ Escalate orders >$5000 to human review
✅ Graceful degradation: show product without recommendations if ML service fails

Customer Support System

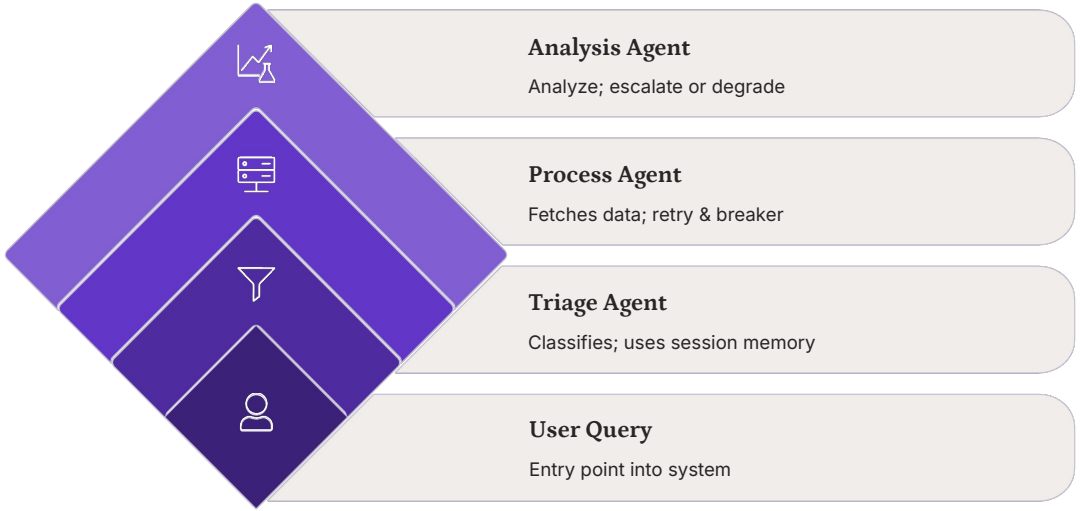
✅ Retry knowledge base search (transient network failure)
✅ Fallback: if KB down, search cached articles
✅ Circuit breaker: if CRM is slow, use cached customer data
✅ Escalate: if agent confidence <60% on answer, escalate to human
✅ Graceful degradation: partial context > no context

Data Processing Pipeline

✅ Retry failed API calls (rate limits)
✅ Circuit breaker: if source API down, skip that source
✅ Graceful degradation: process partial data if one source fails
✅ Alert human: if >30% of data sources fail
✅ Use backoff jitter: prevent all 10 agents retrying simultaneously

Visualizing Complete Workflow with Error Handling

This diagram illustrates a comprehensive multi-agent workflow, integrating various error handling strategies to ensure resilience and a smooth user experience, even when components fail.



Key Principles

- **Each error type has an appropriate strategy:** Tailoring responses to specific failure modes enhances robustness.
- **Cascading failures are prevented:** Mechanisms like circuit breakers isolate issues.
- **User experience degrades gracefully:** Partial results are provided when full functionality isn't possible.
- **Humans are involved for edge cases:** Complex or critical issues are escalated for expert intervention.
- **Monitoring and alerting:** Continuous tracking of error rates allows for proactive issue resolution.

Key Takeaways

As we conclude our session, let's recap the essential insights gained from exploring advanced multi-agent systems.

- **Multi-agent systems enable specialization + modularity:** Design agents for specific tasks, allowing for scalable and maintainable architectures.
- **Three patterns for flow topologies:** Understand when to use Graph (flexible orchestration), Swarm (autonomous cooperation), or Workflow (deterministic sequences) for your agent interactions.
- **GraphBuilder creates explicit workflows:** Use this powerful tool to define clear nodes and edges, ensuring predictable agent behavior.
- **AgentCoreMemorySessionManager for production:** Prioritize robust session management for consistent context retention in live environments, moving beyond simple file-based solutions.
- **Choose error handling pattern based on error type:** Adopt a nuanced approach to errors; a one-size-fits-all solution is insufficient for resilient systems.
- **Exponential backoff + jitter prevents thundering herd:** Implement intelligent retry mechanisms to avoid overwhelming services during temporary outages.
- **Circuit breaker stops cascading failures:** Protect your system from widespread collapse by isolating failing components and preventing continuous requests to unhealthy services.
- **Test error scenarios before production:** Proactive testing of failure modes is crucial for building and deploying robust multi-agent solutions.

Break Time

5 Minute Break ☕

Stretch, grab a coffee, and recharge. When we return, we'll dive into advanced topics that make multi-agent workflows production-ready.

AI Agent Deployment Options: Lambda, Fargate, and AgentCore Runtime

Evaluating three distinct paths to production for AI agents on AWS. Each approach offers unique advantages for different deployment scenarios and team requirements.

Three Paths to Production

Choose the right deployment model based on your agent's requirements, team expertise, and long-term scalability needs.

AWS Lambda

Serverless, auto-scaling

- 15-minute timeout limit
- Pay per invocation
- Instant setup

Best for: Quick integration, legacy microservices, event-driven workloads

AWS Fargate

Container-based orchestration

- Unlimited task duration
- Full infrastructure control
- Custom networking

Best for: Complex workloads, custom orchestration, existing container pipelines

AgentCore Runtime

Purpose-built for AI agents

- 8-hour session support
- Built-in memory & identity
- Execution-time pricing

Best for: Production agents, long workflows, multi-agent systems (GA Oct 2025)

Lambda vs Fargate vs AgentCore: Feature Matrix

Understanding the technical trade-offs helps you select the optimal deployment approach for your agent architecture.

Feature	Lambda	Fargate	AgentCore
Setup Complexity	Simple	Medium	Simple
Max Runtime	15 min	Unlimited	8 hours
Cold Start	Yes (50-200ms)	No	Minimal (<50ms)
Session Management	Manual	Manual	Built-in
Memory Persistence	External	External	Built-in
Cost Model	Per invocation	Per CPU/memory/hour	Execution-time
Multi-Agent Native	❌ No	❌ No	✅ Yes

Workshop Focus: We'll use Lambda for rapid prototyping and learning, but AgentCore is recommended for production agent deployments requiring long-running sessions and built-in orchestration.

AgentCore Cost Advantage: Execution-time pricing means you pay only when agents actively work—not during tool calls, API waits, or user input delays. Lambda charges for every second, even idle time.

Why AgentCore Runtime? Production-Ready Agent Infrastructure

What Makes AgentCore Different

Purpose-built for AI agents from the ground up—not retrofitted from Lambda or container platforms. Launching GA October 2025.

8-hour sessions enable complex workflows without timeout constraints

Built-in memory and identity eliminate custom infrastructure setup

Automatic multi-agent orchestration with native coordination primitives

Session isolation ensures security and compliance by default

Execution-time pricing charges only for active computation

Decision Framework

Choose AgentCore when:

✅ Pure agent deployment without custom business logic
✅ Workflows typically exceed 2 minutes
✅ Multi-agent systems with shared memory
✅ Cross-session context is required

Keep Lambda if:

✅ Existing microservice integration
✅ Sub-1-second response needed (Python SnapStart)
✅ Team expertise in Lambda ecosystem

3s	$0.00003	$0.00002	8hrs
Avg Query Time	**AgentCore Cost**	**Lambda Cost**	**Session Limit**
Typical agent execution duration	Per query with execution-time pricing	Per query (512MB, but timeout risk)	Maximum continuous execution time

Lambda Deployment Architecture

1. **User Request**
   HTTP request via API Gateway endpoint
2. **Lambda Function**
   Executes agent code with Strands SDK integration
3. **Bedrock API**
   Claude or Nova model inference for reasoning
4. **Tool Execution**
   External APIs, databases, and services invoked
5. **Response & Logs**
   CloudWatch captures metrics; user receives result

Benefits

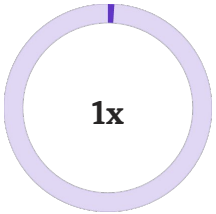
- Zero server management—fully serverless
- Auto-scales to 1,000 concurrent executions
- Pay only for invocations and compute time
- Deep AWS ecosystem integration (IAM, CloudWatch)

Limitations

- 15-minute timeout insufficient for long agents
- Cold start latency on first invocation (50-200ms)
- Manual session management via external storage
- No native memory or identity primitives

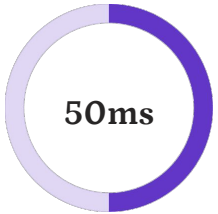
Lambda Handler Pattern: Initialization Best Practice

Critical optimization: Initialize your agent **outside** the handler function to enable warm reuse across invocations, dramatically reducing latency and costs.



Model Initialization
Happens once on first invocation

Cold Start Before
Initial latency with inline initialization



Cold Start After
Reduced latency with warm reuse pattern

Lambda Deployment Process

Five essential steps to take your agent from code to production. Our automated script handles the entire workflow for you.

1

Package Dependencies

Install requirements, create deployment ZIP with Lambda layer structure

2

Create IAM Role

Establish execution role with Bedrock and CloudWatch permissions

3

Deploy Function

Upload code bundle, configure runtime (Python 3.12), set timeout/memory

4

Create Function URL

Generate public HTTPS endpoint for direct invocation

5

Test Invocation

Validate deployment with sample query via curl or API client

Automated Deployment: Run `./deploy.sh` to execute all steps automatically. The script creates IAM roles, packages dependencies, deploys the function, and outputs your ready-to-use API endpoint in under 2 minutes.

IAM Permissions: Least Privilege Principle

Secure your Lambda agent by granting only the minimum permissions required for operation. Avoid wildcards and scope resources precisely.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["bedrock:InvokeModel"],
      "Resource": "arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-3-5-haiku-*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:us-east-1:ACCOUNT:log-group:/aws/lambda/workshop-agent:*"
    },
    {
      "Effect": "Allow",
      "Action": ["secretsmanager:GetSecretValue"],
      "Resource": "arn:aws:secretsmanager:us-east-1:ACCOUNT:secret:agent-api-keys-*"
    }
  ]
}
```

Best Practices

✅ Restrict to specific model families, not all foundation models
✅ Scope log permissions to function's log group only
✅ Use explicit resource ARNs instead of wildcards
✅ Create separate permission blocks per integration
✅ Regularly audit and remove unused permissions

Security Principle: Each permission should have a clear business justification. Never grant "*" actions—always specify exact operations needed.

Hands-On Exercise: Deploy Your Agent to Lambda

Review lambda_handler.py

Examine agent initialization, input/output formatting, and error handling patterns

Run ./deploy.sh

Automated script packages dependencies, creates IAM role, deploys function, and generates API endpoint

Test with curl

Send a POST request with JSON payload to validate agent response and execution

View CloudWatch Logs

Watch real-time execution logs, tool invocations, and performance metrics

Example Test Command:

```
curl https://YOUR_FUNCTION_URL -X POST -d '{"query":"What is 15 * 23?"}' -H 'Content-Type: application/json'
```

Success Criteria

✅ Deployment completes without errors
✅ API endpoint returns JSON response
✅ Logs appear in CloudWatch within 30 seconds
✅ Agent successfully invokes tools and returns results

Production Monitoring with CloudWatch

Logs (Debugging)

- Agent invocations and queries
- Tool execution details
- Error stack traces
- Performance timing

Metrics (Dashboards)

Invocations: Request count

Duration: Execution time (ms)

Errors: Failed requests

Throttles: Concurrency limits

Alarms (Proactive)

- Alert if error rate > 5%
- Alert if duration > 10s
- Alert if daily cost > $10
- Alert on throttling events

1,234	0.24%	2,400ms	$2.15
Total Invocations	**Error Rate**	**Avg Duration**	**Daily Cost**
24-hour rolling window	3 errors out of 1,234 requests	Mean execution time per invocation	Compute charges for past 24 hours

Live Demo: Watch CloudWatch logs update in real-time as we invoke the agent, showing request flow from API Gateway through Bedrock model inference, tool execution, and final response delivery. Use CloudWatch Insights queries to filter errors, analyze performance patterns, and track cost trends over time.

Security Best Practices for Lambda Agents

Implement robust security measures to protect your agent, data, and infrastructure from common vulnerabilities.

Input Validation

Sanitize and validate all incoming requests to prevent malicious input, such as injection attacks or oversized payloads.

```
# Validate query length and content
if len(query) > 5000:
    return {'statusCode': 400}
if any(bad in query for bad in ['eval', 'exec']):
    return {'statusCode': 400}
```

Secrets Management

Store sensitive credentials (API keys, database passwords) in a dedicated secrets management service like AWS Secrets Manager.

```
# Fetch API key from Secrets Manager
import boto3
secrets = boto3.client('secretsmanager')
api_key =
secrets.get_secret_value(SecretId='agent-api-keys')['SecretString']
```

Rate Limiting

Control invocation frequency to prevent abuse, protect backend services, and manage costs. Utilize API Gateway or in-code logic.

```
# API Gateway throttling settings:
# Rate: 100 req/sec, Burst: 1000 req
# Or use DynamoDB for custom in-code rate limiting.
```

CORS Headers

Configure Cross-Origin Resource Sharing (CORS) to specify which domains are allowed to interact with your Lambda function's API endpoint.

```
# Example CORS headers for API Gateway:
'Access-Control-Allow-Origin': 'https://yourdomain.com',
'Access-Control-Allow-Methods': 'POST',
'Access-Control-Allow-Headers': 'Content-Type'
```

Scaling & Performance Optimization

Optimize your Lambda-based AI agents for high performance and cost-efficiency in production.

Horizontal Scaling (Concurrency)

- Lambda auto-scales to 1,000 concurrent executions (default)

Set **reserved concurrency** for critical functions (guarantees capacity)

Monitor **throttling metrics** when function exceeds limits

Example: Reserve 100 concurrent for a production agent

Vertical Scaling (Memory/CPU)

Increase memory: 128MB → 10GB

More memory = proportionally more CPU

- Reduces execution time for faster inference
- Find optimal size by thorough testing

Cold Start Reduction

SnapStart: Reduces to 50ms (Python GA June 2025)

ARM64 architecture: 13-24% faster than x86

Lambda Layers: Reduce deployment package size

Provisioned Concurrency: Eliminates all cold starts (higher cost)

Cost Optimization

Use **ARM64 architecture:** 30% cheaper than x86, faster cold starts

Optimize **memory size:** Better cost/performance ratio

Cache responses: Avoid redundant API calls

Appropriate timeout: 30 seconds usually sufficient for agents

Cold Start Reduction Strategies

Mitigate the impact of cold starts on your Lambda agent's responsiveness using these key strategies.

SnapStart for Python

Pre-initializes function code and runtime snapshot to reduce cold start times significantly (GA June 2025).

```
# Enable on function version (not $LATEST)
aws lambda publish-version --function-name workshop-agent

# Result: Cold start reduced from ~800ms to ~50ms
# Cost: Small snapshot storage fee (~$0.50/month)
```

ARM64 Architecture

Leverage Graviton processors for improved performance and cost efficiency.

```
aws lambda create-function --function-name workshop-agent \
  --architectures arm64 \
  ...
```

Cold start: 13-24% faster than x86
Compute cost: 30% lower vs x86
Trade-off: Some legacy packages may not support ARM64

Lambda Layers

Separate common dependencies from your deployment package to reduce size and improve cold start times.

```
# Create and publish layer
pip install -r requirements.txt -t python/lib/python3.12/site-packages/
zip -r layer.zip python/
aws lambda publish-layer-version --layer-name agent-dependencies \
  --zip-file fileb://layer.zip --compatible-runtimes python3.12

# Attach to function
aws lambda update-function-configuration \
  --function-name workshop-agent \
  --layers arn:aws:lambda:REGION:ACCOUNT:layer:agent-dependencies:1
```

Result: 60-70% faster cold starts for smaller code packages.

Provisioned Concurrency

Keeps functions warm, eliminating cold starts entirely, but at a higher cost.

```
aws lambda put-provisioned-concurrency-config \
  --function-name workshop-agent \
  --provisioned-concurrent-executions 10 \
  --qualifier LIVE
```

Cost: ~$0.000004 per GB-second (e.g., ~$18/day for 100 concurrent 1536MB functions)
Use when: Sub-100ms cold start latency is a critical business requirement.
Consider alternatives: If occasional bursts are acceptable, or explore AgentCore Runtime.

Lambda vs. AgentCore: Choosing the Right Deployment

Selecting between AWS Lambda and AgentCore for your AI agent deployment depends on your specific use case, operational requirements, and long-term strategy.

LAMBDA

When to Deploy Agent to Lambda

- Learning and experimentation phases
- Quick integration with existing Lambda infrastructure
- Agent queries typically complete in < 2 minutes
- Custom business logic layer required
- Team has strong expertise with Lambda

AGENTCORE

When to Deploy Agent to AgentCore

- Production-ready agent deployment
- Long-running workflows (>2 minutes)
- Multi-agent systems with shared memory
- Complex orchestration and state management needs
- Benefit from built-in security and compliance features

Cost Comparison Scenarios

100 queries/day, 3s each	0.15	0.12	20%
1000 queries/day, 5s each	2.50	1.80	28%
Long workflows, 30s each	15.00	8.10	46%

AgentCore demonstrates increasingly significant cost savings as agent workflows become longer, particularly when agents spend time waiting on I/O operations (e.g., tool calls, user input).

Deployment Checklist: Before Production

Ensure your AI agent is production-ready by verifying these essential pre-deployment steps across security, monitoring, performance, and testing.

Pre-Deployment Security

- IAM role uses least privilege
- Secrets stored in Secrets Manager (not code)
- Input validation implemented and tested
- Error handling prevents information leakage
- CORS headers configured for your domain

Pre-Deployment Monitoring

- CloudWatch alarms configured (error rate, duration, cost)
- Log group retention set (30 days minimum recommended)
- Dashboard created for key metrics
- Error notifications enabled

Pre-Deployment Performance

- Rate limiting enabled (API Gateway or in-code)
- Lambda memory size optimized (tested different sizes)
- ARM64 architecture enabled (20-30% cost savings)
- SnapStart enabled (cold starts <50ms)
- Lambda Layers configured (if dependencies large)

Pre-Deployment Testing

- Load tested with expected traffic volume
- Error scenarios tested (timeout, invalid input, API failures)
- Concurrent execution limits tested
- Cost estimates validated
- Rollback procedure documented

Troubleshooting Guide

Identify and resolve common issues encountered when deploying and managing your AI agents on AWS Lambda.

Common Issues & Solutions:

"Task timed out after 15.00 seconds"	Long-running agent	Increase timeout to 30s, or use AgentCore Runtime
"Unable to import module"	Dependency missing in zip	Verify all packages in `site-packages/`, test locally first
"AccessDeniedException"	IAM role missing permission	Check role policy includes `bedrock:InvokeModel`
"Cold start latency high" (>500ms)	First invocation overhead	Enable SnapStart (Python 3.12+), use provisioned concurrency, or ARM64
"Cost unexpectedly high"	Inefficient prompts or tools	Optimize system prompt, cache results, reduce API calls
"Function throttled"	Exceeded concurrency limit	Increase reserved concurrency or set up auto-scaling

Debugging with CloudWatch:

```
# Filter for errors
aws logs filter-log-events \
  --log-group-name /aws/lambda/workshop-agent \
  --filter-pattern "ERROR"

# Get last 50 log events
aws logs tail /aws/lambda/workshop-agent --follow --max-items 50
```

Summary: Empowering Your AI Agent Deployments

Review the core insights from our discussion on deploying and managing AI agents effectively.

Lambda: Quick Start & Serverless Agility

Ideal for initial deployment with its serverless nature, ease of use, and a 15-minute execution limit.

AgentCore Runtime: Production-Ready Power

Designed for agent-native workloads, offering extended 8-hour sessions and recommended for robust production environments.

Optimize Initialization

Initialize your agent outside the handler function to enable reuse across multiple invocations, reducing latency and cost.

Leverage CloudWatch

Utilize CloudWatch for comprehensive monitoring, logging, and debugging to maintain agent health and performance.

Robust Security Measures

Implement IAM roles with least privilege, validate inputs rigorously, and secure sensitive data with secrets management services.

Boost Performance & Efficiency

Employ ARM64 architecture, SnapStart, Lambda Layers, and continuous cost optimization strategies for superior performance.

Thorough Pre-Production Testing

Always test error scenarios, load, and concurrency limits before moving your AI agent to a live production environment.

Strategic Platform Selection

Match your deployment platform to your workload requirements: Lambda for rapid prototyping, AgentCore for demanding production agents.

Q&A

Connect

- https://github.com/eduamota/building-apps-with-bedrock
- https://www.linkedin.com/in/motaed/

- Agentic AI and Cybersecurity Risks (live online course with Petar Radanliev)
- AI Agents A-Z (live online course with Sinan Ozdemir)
- AI Memory Management in Agentic Systems (live online course with Richmond Alake)
- *Building Agentic AI Systems* (book)
- *AI Agents: The Definitive Guide* (book)


O'REILLY®